SoundSignal: Predicting Spotify Popularity and Decomposing

Milan Giliazetdinov

Abstract

Predicting a song's Spotify popularity is only half of a useful answer. A single blended score cannot tell a musician whether a track scored well because the artist is already famous, because the genre is commercially favoured, or because of the recording itself. This project trains a gradient-boosted model on 66,808 cleaned tracks and then *decomposes* its prediction into four additive terms: a typical-track baseline, artist fame, genre, and the recording's own contribution. Along the way two instructive failures were documented: a leave-one-out artist-fame feature that silently leaked the target ($\rho = 0.85$ with popularity), and an evaluation setup in which forgetting to remove genre inflated the apparent audio signal roughly 3.5× (0.128 → 0.460 Spearman). The final composed model reaches $R^2 = 0.6206$ and Spearman 0.7917, and the honest measured finding is that the contribution attributable to audio alone is small once fame and genre are accounted for.

1 Introduction

Spotify assigns every track a *popularity* score from 0 to 100. Predicting it is a standard regression problem, but a prediction alone is commercially uninteresting to the person who made the song: it does not separate what the artist's existing audience contributed from what the record contributed.

This project therefore has two goals. The first is conventional — try fit the best model possible. The second is the actual product: split the predicted number into interpretable, additive parts, so that

$$\hat{y} = b + \Delta_{\text{fame}} + \Delta_{\text{genre}} + \Delta_{\text{audio}} \tag{1}$$

holds exactly, and each term can be shown to a user with a defensible meaning.

Two models are trained. A **research model** uses Spotify's own engineered audio features (danceability, valence, energy, and so on) and exists to answer the modelling question as well as possible. A **production model** uses `librosa` descriptors computed directly from an uploaded mp3, because Spotify deprecated public access to its audio-features endpoint and those features are simply unavailable for a user's own file.

2 Dataset and Preprocessing

2.1 The raw data

The starting corpus contains 114,000 Spotify tracks with audio features, genre labels, and popularity (Figure 1, Figure 3). It is not usable as delivered. A large share of rows are duplicate copies of original songs that appeared in user playlists under different track identifiers. These duplicates carry near-zero popularity while being, musically, the same recording as a well-known track. Left in place they teach the model that famous songs are unpopular.

2.2 Building an artist-fame signal

Removing those rows requires knowing whether the *artist* is significant, and Spotify's dataset ships no artist-popularity field. We therefore constructed one by target encoding: for track i by artist a, average the popularity of that artist's *other* tracks,

$$\texttt{artist\_fame\_loo}_i = \frac{1}{|A_i| - 1} \sum_{j \in A_i,\, j \neq i} y_j, \tag{2}$$

where A_i is the set of tracks by artist a. The leave-one-out construction excludes y_i itself, which is what prevents a song's own popularity from flowing directly into its own feature. This worked well for *cleaning*: a track whose artist averages a high popularity but which itself sits at zero is almost certainly a playlist duplicate, and can be dropped. Cleaning reduced 114,000 rows to **66,808**.

It did not work as a *feature*. Equation 2 is still built out of the target variable, and it shows: `artist_fame_loo` correlates with popularity at $\rho = 0.85$ (Figure 5). A model given it reports strong results that are largely the target predicting itself. It is also uncomputable at serving time, since a newly uploaded song has no catalogue of sibling tracks to average.

It was decided to replaced it with an external signal: total listener counts per artist pulled from the Last.fm API. This feature has an honest weakness — a large listener count is not the same thing as fame, since it accumulates over time and rewards longevity as much as current standing — but no better public alternative was available, and critically it correlates with popularity at only $\rho = 0.42$, i.e. it carries information without being a proxy for the answer.

2.3 Scaling

Both `duration_ms` and `artists_listeners` are strongly right-skewed and live on scales orders of magnitude larger than the $[0, 1]$ audio features. We apply $x \mapsto \log(x)$ to both, which compresses the tail onto a comparable range (Figure 2). This matters most for the linear models, where a feature's scale directly affects its coefficient and any regularisation penalty applied to it.

3 Exploratory Data Analysis

Figures 3 and 4 show feature distributions before and after cleaning. Most audio features are close to normally distributed; `speechiness`, `instrumentalness`, `liveness` and `acousticness` are heavily concentrated near zero, which is expected given that most commercially released tracks are studio recordings with sung vocals.

The Spearman correlation matrix (Figure 5) drives the modelling strategy. Against popularity, the two strongest relationships by a wide margin are `artists_listeners` (0.42) and genre; every individual audio feature sits near zero, the largest being `danceability` at 0.11 and `instrumentalness` at -0.19. Among features themselves the structure is musically sensible: energy and loudness move together (0.75), energy and acousticness oppose one another (-0.71), and danceability tracks valence (0.47).

4 Model Selection

It was decided to try several models for both baseline estimation and understanding of structure of data.

4.1 Ordinary least squares

OLS fits a linear map $\hat{y} = X\beta$ by minimising squared error,

$$\hat{\beta} \;=\; \arg\min_{\beta} \|y - X\beta\|_2^2 \;=\; (X^\top X)^{-1} X^\top y, \tag{3}$$

and serves as the baseline. Genre is nominal and must be one-hot encoded, which expands X by 114 columns. Results are in Figure 6: test $R^2 = 0.5983$, MAE 8.269, RMSE 11.310. Train and test scores are nearly identical ($R^2 = 0.6009$ vs 0.5983).

4.2 Lasso

With genre one-hot encoded, most of the design matrix is sparse indicator columns, many of which are likely uninformative. Lasso adds an L_1 penalty,

$$\hat{\beta} \;=\; \arg\min_{\beta} \frac{1}{2n}\|y - X\beta\|_2^2 \;+\; \alpha\|\beta\|_1, \tag{4}$$

whose non-differentiable corner at the origin drives coefficients to *exactly* zero rather than merely shrinking them. It therefore performs implicit feature selection, which is the reason to prefer it here over ridge regression. Under nested cross-validation (Figure 7) it reaches $R^2 = 0.5984$, MAE 8.277, RMSE 11.233.

4.3 Diagnosis: high bias, not high variance

Lasso performs essentially identically to OLS (0.5984 vs 0.5983), and OLS train and test scores barely differ. Regularisation changing nothing means variance is not the limiting factor. Combined with high absolute error, this is the signal of **high bias**. Hence the hypothesis class is: model is too simple to for the data, and hence a more complex model should be utilized.

4.4 Multilayer perceptron

A neural network with one hidden layer computes

$$h = \sigma(W_1 x + b_1), \qquad \hat{y} = W_2 h + b_2, \tag{5}$$

where the non-linear activation σ allows curved decision surfaces that no linear model can express. It improves notalby (Figure 8): test $R^2 = 0.6494$, MAE 7.285, RMSE 10.540. The gap between train $R^2 = 0.7284$ and test 0.6494 indicates mild overfitting, but the the hypothesis is proven: **the data has non-linear relationship**. Given that `artists_listeners` is the dominant numeric predictor, this implies the fame-to-popularity relationship is curved. This makes a perfect sense, since the difference between 10,000 and 1,000,010 listeners matters far more than between 5M and 6M.

That conclusion is what motivates tree ensembles, which capture non-linearity and interactions natively and handle the mixed continuous/categorical feature space without one-hot encoding.

4.5 Tree ensembles

Totally the performance of four ensembles (Figure 9) was evaluated. **Random forests** average B decorrelated trees grown on bootstrap samples,

$$\hat{y} = \frac{1}{B}\sum_{b=1}^{B} T_b(x), \tag{6}$$

reducing variance through averaging. **Extremely randomised trees** go further and draw split thresholds at random, trading a little extra bias for another variance reduction. Both are *bagging* methods: trees are grown independently and in parallel.

Gradient boosting is structurally different. It fits trees in sequence, each one correcting its predecessors' errors:

$$F_m(x) \;=\; F_{m-1}(x) \;+\; \nu\, h_m(x), \tag{7}$$

	track_id	popularity	duration_ms	danceability	energy	key	loudness	mode
count	114000.000000	114000.000000	1.140000e+05	114000.000000	114000.000000	114000.000000	114000.000000	114000.000000
mean	56999.500000	33.238535	2.280292e+05	0.566800	0.641383	5.309140	-8.258960	0.637553
std	32909.109681	22.305078	1.072977e+05	0.173542	0.251529	3.559987	5.029337	0.480709
min	0.000000	0.000000	0.000000e+00	0.000000	0.000000	0.000000	-49.531000	0.000000
25%	28499.750000	17.000000	1.740660e+05	0.456000	0.472000	2.000000	-10.013000	0.000000
50%	56999.500000	35.000000	2.129060e+05	0.580000	0.685000	5.000000	-7.004000	1.000000
75%	85499.250000	50.000000	2.615060e+05	0.695000	0.854000	8.000000	-5.003000	1.000000
max	113999.000000	100.000000	5.237295e+06	0.985000	1.000000	11.000000	4.532000	1.000000

	speechiness	acousticness	instrumentalness	liveness	valence	tempo	time_signature
count	114000.000000	114000.000000	114000.000000	114000.000000	114000.000000	114000.000000	114000.000000
mean	0.084652	0.314910	0.156050	0.213553	0.474068	122.147837	3.904035
std	0.105732	0.332523	0.309555	0.190378	0.259261	29.978197	0.432621
min	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
25%	0.035900	0.016900	0.000000	0.098000	0.260000	99.218750	4.000000
50%	0.048900	0.169000	0.000042	0.132000	0.464000	122.017000	4.000000
75%	0.084500	0.598000	0.049000	0.273000	0.683000	140.071000	4.000000
max	0.965000	0.996000	1.000000	1.000000	0.995000	243.372000	5.000000

Figure 1: Descriptive statistics of the raw 114,000-track dataset. Popularity averages 33.2 with a first quartile of 17, pulled down by the playlist duplicates; `duration_ms` spans four orders of magnitude, motivating the log transform.

	Mean	Std Dev	Min	Median	Max
popularity	38.776554	17.727055	0.0	39.0	100.0
duration_ms	12.267987	0.359186	11.002266	12.27311	13.304685
acousticness	0.327854	0.336405	0.0	0.194	0.996
danceability	0.561716	0.174577	0.0513	0.575	0.984
energy	0.64007	0.254949	0.00002	0.682	1.0
instrumentalness	0.176523	0.325984	0.0	0.000069	1.0
liveness	0.2213	0.200004	0.00925	0.133	1.0
loudness	-8.497922	5.207704	-46.591	-7.2095	4.532
speechiness	0.091076	0.121269	0.0221	0.0495	0.963
tempo	122.335878	29.725401	41.858	122.0065	243.372
time_signature	3.907556	0.420006	0.0	4.0	5.0
valence	0.469159	0.263081	0.0	0.456	0.995
artists_listeners	11.352514	2.215523	0.693147	11.576229	16.036949

Figure 2: Descriptive statistics after cleaning (66,808 tracks). Note `duration_ms` and `artists_listeners` are log-transformed.

where h_m is fitted to approximate the negative gradient of the loss, $-\partial\mathcal{L}(y, F_{m-1})/\partial F_{m-1}$, and ν is the learning rate. Because each tree targets the residual error left by the ensemble so far, boosting builds far more expressive functions than an average of independent trees.

The results follow that ordering: RandomForest is worst ($R^2_{\text{holdout}} = 0.5865$), the two boosting variants and ExtraTrees cluster around 0.64–0.65, and **LightGBM** — a gradient booster using leaf-wise tree growth and histogram-binned features — performed best overall, reaching $R^2 = 0.7067$, MAE 6.537 and Spearman 0.8411 (Figure 10).

Its feature importances make the central finding of the whole project explicit: `track_genre` (1863) and `artists_listeners` (1614) dominate, while every audio feature lands between 471 and 618. The conclusion here is that the context rather than sound, explains most of popularity.

5 From Spotify Features to librosa

The research model above cannot be deployed. Spotify deprecated public access to its audio-features endpoint, so when a user uploads an mp3 there is no way to obtain `danceability` or `valence` for it. Two options existed.

Option A was to estimate Spotify's features from signal-level descriptors — train a model per feature, then feed the estimates into the existing pipeline. Its appeal is interpretability, since the final explanation could keep using familiar names. It was decided to reject this idea. Spotify's features are themselves the output of proprietary models almost certainly more sophisticated than a linear or tree regressor; approximating them with a simpler model degrades the signal, and that degraded estimate is then consumed by a second model whose own error compounds it. The layer cannot add information that the signal-level descriptors did not already carry.

Option B, which was adopted, is to retrain the audio model directly on `librosa` descriptors extracted from the waveform: tempo, onset rate, RMS energy and dynamic range, spectral centroid, bandwidth, rolloff, flatness and contrast, zero-crossing rate, MFCCs, and chroma statistics.

The empirical result vindicates the choice. On identical rows, the librosa audio model reaches a residual Spearman of **0.128** against **0.125** for Spotify's own engineered features, with a shuffled-feature control at 0.008 (Figure 11, left). Speaking librosa costs essentially nothing.

6 The Serving Pipeline

6.1 Why Spearman

Spearman's rank correlation was used

$$\rho \;=\; \operatorname{corr}\big(\operatorname{rank}(y), \operatorname{rank}(\hat{y})\big), \tag{8}$$

as a headline metric alongside MAE, RMSE and $R^2 = 1 - \sum(y_i - \hat{y}_i)^2 / \sum(y_i - \bar{y})^2$. The reason is that Spotify's popularity value is an abstract, recency-weighted index whose absolute magnitude is hard to interpret, whereas *ordering* songs correctly is directly meaningful and is what the product actually presents.

6.2 The key insight: genre is audio

Audio features alone yield only about 0.13 Spearman on the residual, and the LightGBM importances put genre and fame far ahead. At first it might seem that the audio does not matter – but this is wrong assumption.

Genre *is* audio. A genre label is essentially a name attached to a region of audio-feature space: acousticness separates classical, energy and distortion separate metal,

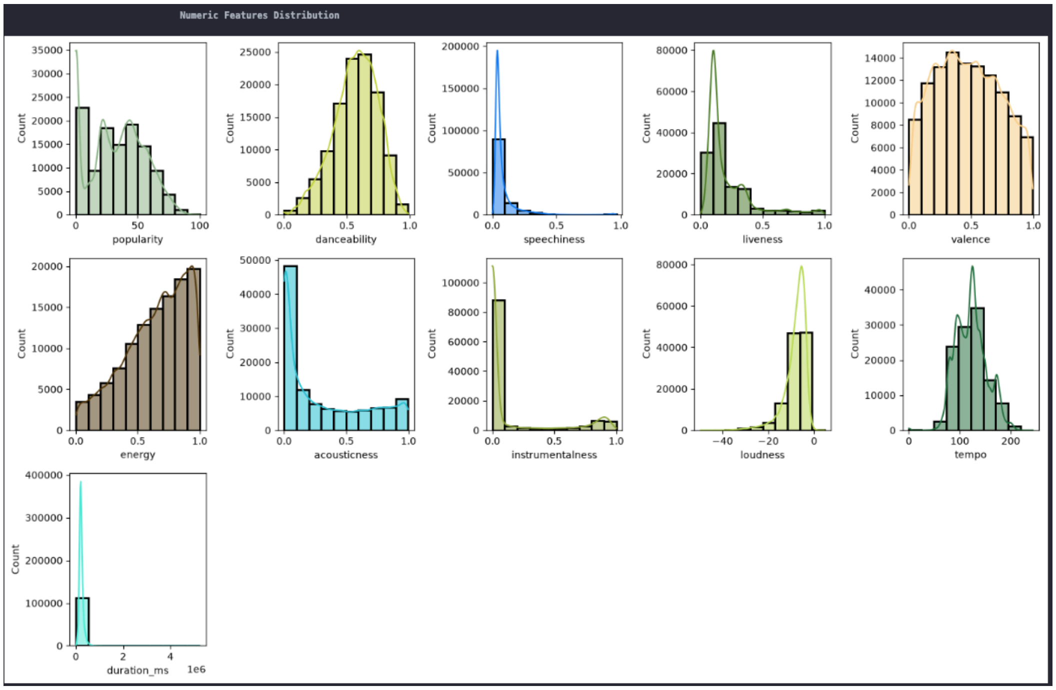

Figure 3: Numeric feature distributions in the raw 114,000-track dataset. Popularity has a large spike at zero — the playlist duplicates.

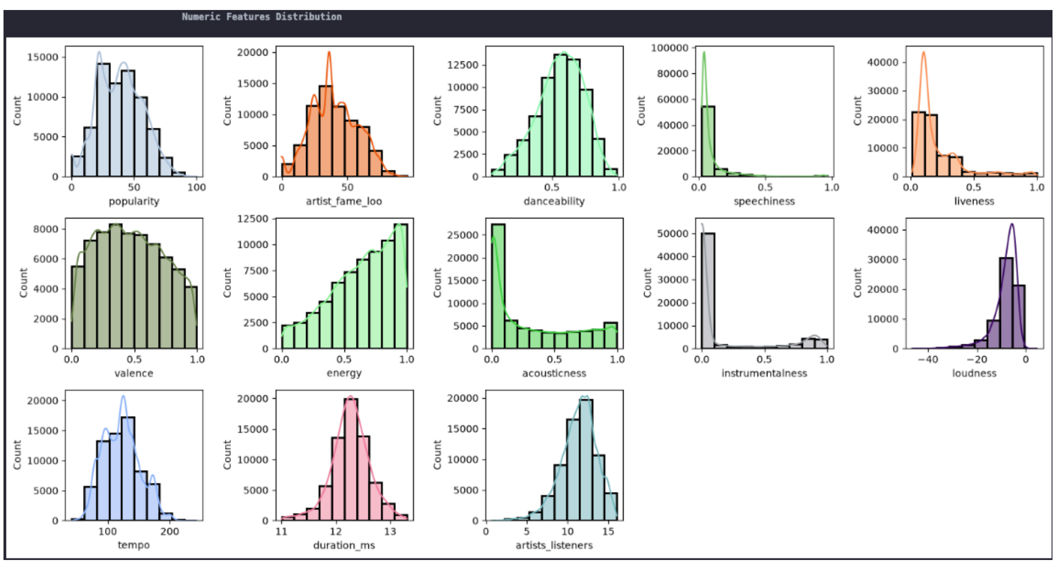

Figure 4: Numeric feature distributions after cleaning. The zero-popularity spike is gone and popularity is approximately normal.

speechiness separates rap. When genre is included as a feature it absorbs that shared information first, leaving audio only whatever is left over.

This is measurable. Removing genre from the context model, so that only fame is residualised out, raises the audio model's Spearman from 0.128 to **0.460** for librosa and from 0.125 to 0.429 for the Spotify features — roughly a 3.5× inflation (Figure 11, right). The apparent gain is not new signal; it is the audio model re-identifying genre.

This cuts both ways, and both directions matter. It proves audio genuinely carries popularity-relevant information. It also proves that an evaluation which forgets to remove genre will dramatically overstate the audio model's ability to judge a *song*, and that comparing a pop track's audio against a classical track's is not a fair comparison. The decomposition below is therefore constructed per genre.

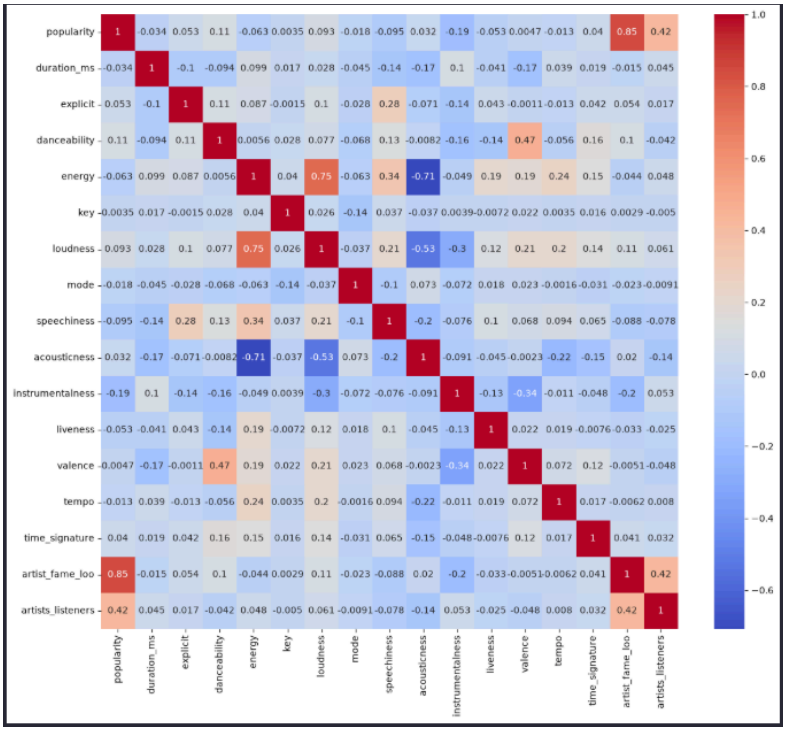

Figure 5: Spearman correlation matrix. `artist_fame_loo` correlates 0.85 with popularity — the target leak that forced its removal — against 0.42 for the external `artists_listeners` replacement.

```
Train R2: 0.6008513063358529
Test R2: 0.5983075798744264
Train MAE: 8.25245448417578
Test MAE: 8.26888608140734
Train RMSE: 11.181045836366748
Test RMSE: 11.309977016968473
```

Figure 6: OLS on Spotify features. Train and test are nearly identical.

```
Nested CV R2: 0.5983513394650227
Nested CV MAE: 8.276601252583458
Nested CV RMSE: 11.23331729349871
```

Figure 7: Lasso under nested cross-validation — statistically indistinguishable from OLS, indicating bias rather than variance is the bottleneck.

```
Train R2: 0.7284288049849075
Test R2: 0.6494341734710606
Train MAE: 6.409218320956863
Test MAE: 7.284589395368594
Train RMSE: 9.22853708139005
Test RMSE: 10.539666082292202
```

Figure 8: MLP results. The jump over the linear models is evidence of a non-linear fame-to-popularity relationship.

	model	cv_rmse	cv_r2	holdout_rmse	holdout_mae	holdout_r2
3	XGBRegressor	10.6031	0.6390	10.6056	7.4617	0.6539
2	ExtraTreesRegressor	10.8370	0.6230	10.6340	7.3775	0.6521
0	HistGradientBoostingRegressor	10.7040	0.6321	10.8151	7.7801	0.6401
1	RandomForestRegressor	11.6243	0.5662	11.5921	8.3391	0.5865

Figure 9: Tree ensemble comparison under cross-validation and on a holdout set. The boosting methods and ExtraTrees cluster around $R^2 \approx 0.64$–0.65; the plain random forest trails at 0.5865.

6.3 Two-model residual architecture

The production pipeline is deliberately built from two models rather than one large regressor. A single model given fame, genre, and audio features could produce a strong popularity estimate, but every feature would compete to explain the same target. Its output would not reveal whether the prediction came from an artist's existing audience or from the recording. The two-model design instead assigns the two questions separate roles.

The **context model** f_c is a LightGBM regressor trained on all 66,808 cleaned tracks. It receives the artist's Last.fm listener count L and the track genre g,

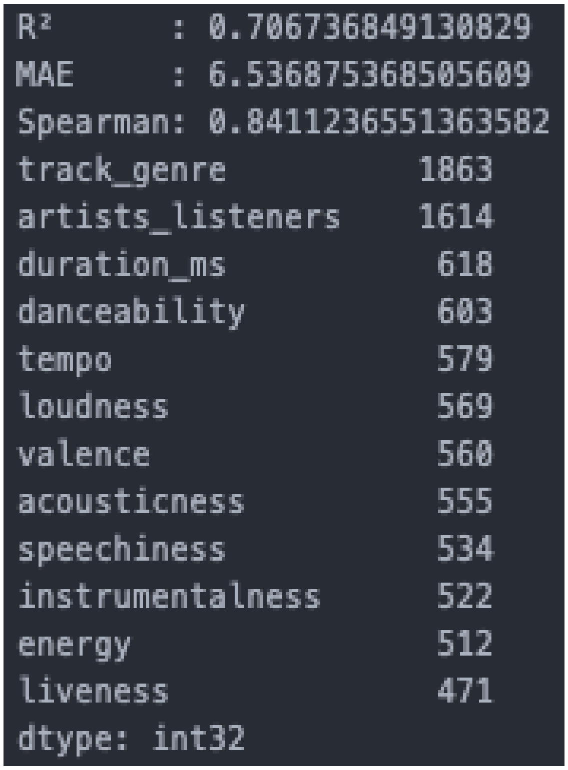
```
R²      : 0.706736849130829
MAE     : 6.536875368505609
Spearman: 0.8411236551363582
track_genre          1863
artists_listeners    1614
duration_ms           618
danceability          603
tempo                 579
loudness              569
valence               560
acousticness          555
speechiness           534
instrumentalness      522
energy                512
liveness              471
dtype: int32
```

Figure 10: LightGBM: best overall, and its feature importances show genre and artist listeners dominating every audio feature.

and predicts the popularity expected from those surrounding conditions:

$$\hat{y}_{\text{context}} = f_c(L, g). \tag{9}$$

This is the appropriate starting point for the score. A new release by an established pop artist and an unsigned classical release do not begin with the same commercial expectation, regardless of their audio.

The **audio model** f_a is a second LightGBM regressor. It never receives the artist name, listener count, or genre as an input. Its inputs are the 58 `librosa` descriptors z extracted from the uploaded waveform, and its training target is the part of popularity left unexplained by the context model:

$$r_i = y_i - \hat{y}_{c,i}^{(-k)}. \tag{10}$$

Consequently, $f_a(z)$ is a signed adjustment of a song in a given context: positive when the recording contains patterns associated with performing above its contextual expectation, negative when it is associated with performing below it.

Training proceeds sequentially. First, group-aware out-of-fold context predictions are generated for the downloaded-audio subset. These predictions are subtracted from the observed popularity to form the residual target in Equation 10. The audio model is then trained to predict that residual from the waveform descriptors. The out-of-fold step is essential: residuals from a context model evaluated on its own training rows would be biased toward zero and would give the audio model an artificially easy, distorted target.

At serving time the two branches run independently and their outputs are added:

$$\begin{aligned} (L, g) &\xrightarrow{f_c} \hat{y}_{\text{context}}, \\ \text{mp3} &\xrightarrow{\texttt{librosa}} z \xrightarrow{f_a} \hat{y}_{\text{audio}}, \\ \hat{y} &= f_c(L, g) + f_a(z). \end{aligned} \tag{11}$$

The logic is therefore: *given who released this track and the market in which it competes, what popularity should be expected; then, how does this particular recording move that expectation?* Keeping genre in the first branch is crucial. If it were left in the residual, the audio model could earn apparent accuracy simply by recognising genre, rather than by distinguishing recordings within that genre.

This separation gives up some of the predictive freedom of an unconstrained blended model, but it makes the audio output defensible and actionable. It also explains the structure of the score shown to the user. The system still contains only two learned models: the next subsection algebraically divides the context output into the typical-track baseline, artist fame, and genre, then recentres the residual output to obtain the audio term. It does not train four independent predictors.

6.4 The four-term decomposition

Let L denote an artist's listener count, g a genre, G the set of training genres, f_c the context model (fame and genre), f_a the audio model, and z a track's librosa features. Define the *marginal* context prediction at fame level L, with genre integrated out:

$$m(L) \;=\; \frac{1}{|G|} \sum_{g \in G} f_c(L, g). \tag{12}$$

Typical track. The baseline is the marginal prediction at the median listener count $\tilde{L}$, i.e. a median-fame artist in an average genre:

$$b \;=\; m(\tilde{L}). \tag{13}$$

Artist fame. How much this artist's audience moves the score away from that baseline, with genre still averaged out so the term isolates fame:

$$\Delta_{\text{fame}} \;=\; m(L) - b. \tag{14}$$

Genre. How much *this* genre is worth at *this* artist's level of fame, plus a correction o_g defined below:

$$\Delta_{\text{genre}} \;=\; f_c(L, g) - m(L) + o_g. \tag{15}$$

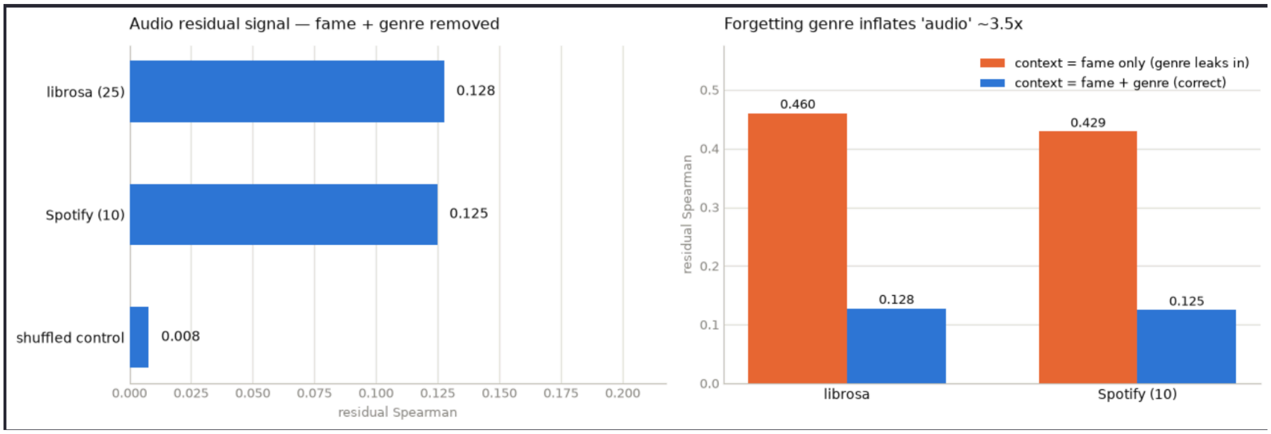

Figure 11: Left: with fame *and* genre removed, librosa (0.128) and Spotify's engineered features (0.125) are near-identical, both far above the shuffled control (0.008). Right: residualising on fame alone leaves genre in the target and inflates the apparent audio signal roughly 3.5×.

Audio. The audio model is trained not on popularity but on the *residual* — what fame and genre could not explain:

$$r_i \;=\; y_i - \hat{y}_{c,i}^{(-k)}, \tag{16}$$

where $\hat{y}_{c,i}^{(-k)}$ is an out-of-fold context prediction. The recording's contribution is then the audio model's output minus the same per-genre offset:

$$\Delta_{\text{audio}} \;=\; f_a(z) - o_g. \tag{17}$$

The per-genre offset. Even after residualising, the audio model's output retains a mild genre bias, because the context model's removal of genre is imperfect. The offset is the mean out-of-fold audio prediction within genre g,

$$o_g \;=\; \frac{1}{|S_g|} \sum_{i \in S_g} \hat{r}_i^{(-k)}, \tag{18}$$

over the set S_g of calibration tracks in that genre. Subtracting it from the audio term and adding it to the genre term moves that systematic component to where it belongs. Since the same quantity is subtracted from one term and added to another, the total in Equation 1 is unchanged — the decomposition still sums exactly to the prediction, but the audio term now measures deviation *within* a genre rather than membership *of* one.

6.5 Out-of-fold estimation

Both the residual target and the offset are computed out-of-fold: the data is partitioned into k folds, and each sample is scored by a model trained without it. An in-sample prediction is partly memorised, which biases residuals toward zero and would corrupt the audio model's target. The context model uses a 10-fold `GroupKFold` split grouped by primary artist, so no artist appears in both training and validation — otherwise the model can learn artist-specific residuals, reintroducing exactly the fame effect the residualisation was designed to remove.

6.6 Calibration

A raw contribution of +2.1 points does not tell a user whether +2.1 is good. We therefore calibrate the audio term into a within-genre percentile:

1. Predict popularity from context (fame and genre).
2. Compute residuals $r_i = y_i - \hat{y}_{c,i}^{(-k)}$.
3. Train the audio model on those residuals.
4. Produce an out-of-fold audio prediction for every calibration track.
5. Build a 101-point percentile grid per genre from those predictions.
6. Compute the per-genre offset o_g.

The grid is built from *predictions* rather than true residuals because a model with modest explanatory power correctly hedges toward the mean: its outputs span a far narrower range than the target does. Ranking predictions against the true-residual spread would compress every song into the middle of the scale. Genres with too few calibration tracks fall back to the global grid.

6.7 Composed performance

Figure 12 gives the final ablation. Adding audio to fame and genre improves every metric — MAE 7.708 → 7.694, RMSE 11.026 → 11.013, R^2 0.6198 → 0.6206, Spearman 0.7880 → 0.7917. The improvement is real and consistent, and it is small. That is the honest result: once an artist's reach and a track's genre are known, the recording itself adjusts the expected popularity by a few points.

Model	MAE ↓	RMSE ↓	R² ↑	Spearman ↑
Fame + genre only	7.708	11.026	0.6198	0.7880
Audio + fame + genre	**7.694**	**11.013**	**0.6206**	**0.7917**

Figure 12: Composed pipeline. Audio improves every metric over fame and genre alone, by a small and consistent margin.

7 Explanation Layer

The decomposition is only useful if a musician can read it, and terms like "MFCC 7" or "spectral flatness" are not readable. The final stage converts the numbers into plain English.

Attribution uses SHAP values, which distribute a prediction across its features according to the Shapley value from cooperative game theory:

$$\phi_j = \sum_{S \subseteq F \setminus \{j\}} \frac{|S|!\,(|F|-|S|-1)!}{|F|!} \big[f(S \cup \{j\}) - f(S)\big]. \quad (19)$$

The property that makes this usable is additivity:

$$f(x) \;=\; \phi_0 + \sum_j \phi_j, \quad (20)$$

so the parts provably sum to the whole. Because SHAP values are additive, related descriptors can be summed into groups without breaking that guarantee — the 58 librosa descriptors are folded into 12 human-readable concepts such as brightness, note density, dynamic range and overall timbre. This is also the only honest way to discuss MFCCs, since no individual coefficient has a meaning worth reporting.

Those grouped values are serialised to JSON and passed to a locally hosted large language model, together with a predefined glossary explaining what each concept means. The model's role is strictly translation: it verbalises numbers that were already computed, and it neither predicts nor attributes anything itself.

8 Conclusion and Limitations

The composed model reaches $R^2 = 0.6206$ and Spearman 0.7917, and decomposes every prediction into four terms that sum to it exactly.

Three limitations deserve statement. First, the target is noisy: Spotify's popularity index is a recency-weighted black box, so small differences between models should not be over-read. Second, `artists_listeners` measures cumulative reach rather than current fame, and rewards longevity. Third, and most importantly, the audio contribution is genuinely small once fame and genre are removed — and the $0.128 \to 0.460$ result shows precisely how easy it would be to report a much more flattering number by residualising on fame alone. The project's central methodological claim is that resisting that temptation is what makes the remaining number trustworthy.